SaaS Retention Analysis

A SAAS BUSINESS CASE STUDY ON CUSTOMER RETENTION, PRODUCT ENGAGEMENT, AND REVENUE RISK

OPERATION RAVENSTACK

Uncovering the Churn Crisis Before Public Launch

Dataset : Synthetic SaaS Pilot Data
Tool : MySQL
Tables : 5 | Rows: 30,000+

A stealth AI startup Built for coding bootcamp graduates.

Every signup tracked.

Every feature click logged.

Every support ticket recorded.

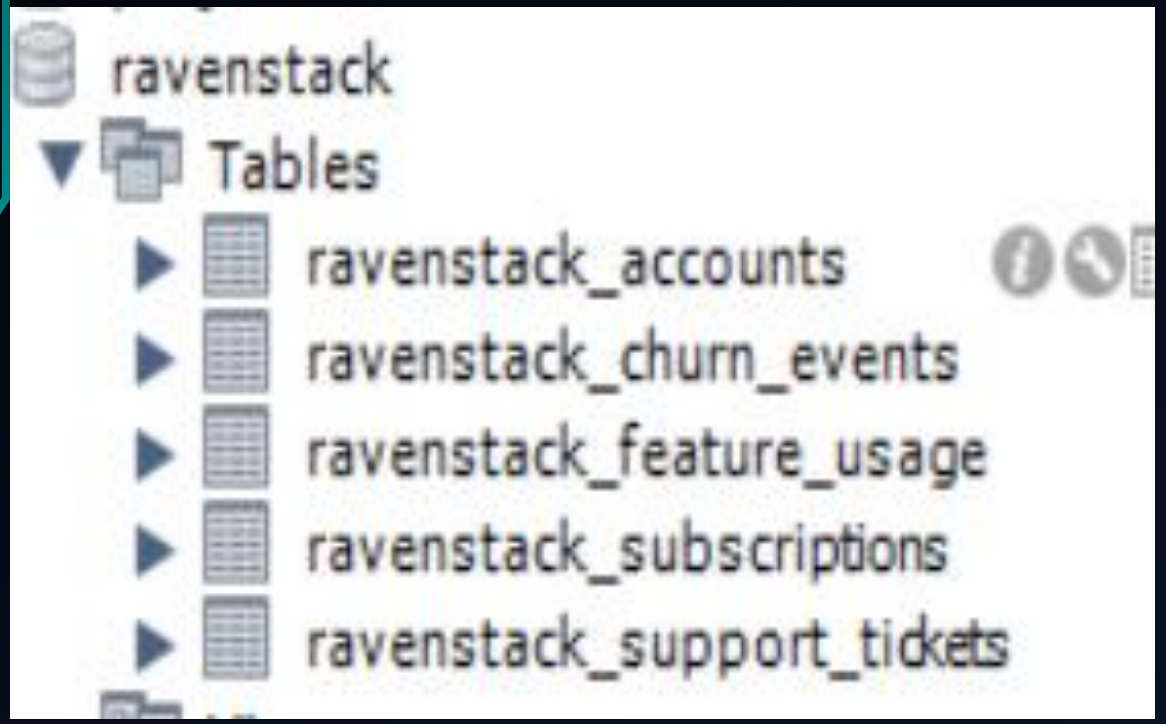
Every goodbye documented.

3 days before public launch and the founders asked one question:

"Is our product ready —or is there a crisis in the data?"

Dataset Overview

TABLE	ROWS	INFO
accounts	500	Every pilot user
subscription	5000	Their plan details
feature_usage	25000	What they used
Churn_events	600	Every Goodbye
Support_tickets	2000	Request for help



Before Analysis — Data Was Cleaned

Issues

- Boolean columns stored as text true/false instead of 1/0 affected: churn_flag, is_trial in accounts and subscriptions.
- Numeric columns stored as text mrr_amount, arr_amount, satisfaction_score, resolution_time_hours
- Date columns in wrong format start_date, end_date, churn_date
- Csv columns loaded in wrong order subscriptions table needed full reload with correct mapping

Fixes

- Created _clean columns for all boolean and numeric columns
- Used LIKE '%rue%' to convert True → 1 and False → 0
- Used CAST to convert text numbers to DECIMAL
- Used STR_TO_DATE to fix all date columns
- Reloaded subscriptions with correct column order

How many customer accounts have churned?

110 OUT OF 500 ACCOUNTS HAVE CHURNED (22% CHURN RATE)

Healthy SaaS benchmark: 5%
Ravenstack is 4x above benchmark

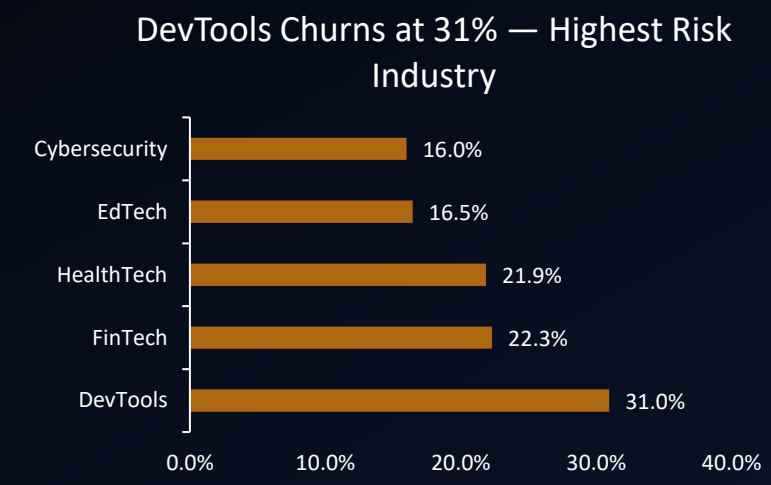
```
select
    count(*) as total_accounts,
    sum(churn_flag_clean) as churned_accounts,
    round(sum(churn_flag_clean) * 100.0 / count(*), 1) as churn_rate_pct
from ravenstack_accounts;
```

OUTPUT:

total_accounts	churned_accounts	churn_rate_pct
500	110	22.0

DevTools Industry is the Crisis

DevTools: 31% churn rate
Average: 22% churn rate
DevTools churns at nearly 1.5x the average
Needs immediate customer success attention




```sql
Select
    industry,
    count(*) as total_accounts,
    SUM(churn_flag_clean) as churned,
    round(sum(churn_flag_clean) * 100.0 / count(*), 1) AS churn_rate_pct
from ravenstack_accounts
where industry is not null
group by industry
order by churn_rate_pct DESC;
```

OUTPUT:

industry	total_accounts	churned	churn_rate_pct
DevTools	113	35	31.0
FinTech	112	25	22.3
HealthTech	96	21	21.9
EdTech	79	13	16.5
Cybersecurity	100	16	16.0

Billing Frequency: A Surprising Result

Expected finding: Annual = lower churn
Actual finding: Difference is only 1.3 points
What this tells us: Billing frequency is not the main churn driver at RavenStack
The problem runs deeper than billing

```sql
select
    billing_frequency,
    count(*) as total_subscriptions,
    sum(churn_flag_clean) as churned,
    round(sum(churn_flag_clean) * 100.0 / count(*), 1) as churn_rate_pct,
    round(avg(mrr_amount), 0) as avg_mrr
from ravenstack_subscriptions
where is_trial_clean = 0
and mrr_amount > 0
and billing_frequency is not null
group by billing_frequency
order by churn_rate_pct desc;
```

OUTPUT:

billing_frequency	total_subscriptions	churned	churn_rate_pct	avg_mrr
annual	2087	216	10.3	2682
monthly	2135	192	9.0	2689

Why are they leaving?

Features + support = 36.3% of all exits

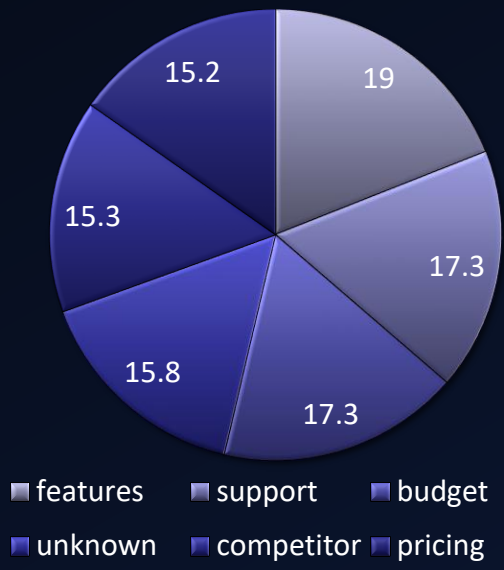
These are INTERNAL problems. Not competitor pressure, not market conditions, not pricing alone. Internal problems have internal solutions.

Both are fixable before launch

```sql
select
    coalesce(reason_code, 'Not Stated') as reason_code,
    count(*) as total_events,
    round(count(*) * 100.0 /
        (select count(*) from ravenstack_churn_events), 1) as percentage
from ravenstack_churn_events
group by reason_code
order by total_events desc;
```

OUTPUT:

	reason_code	total_events	percentage
▶	features	114	19.0
	support	104	17.3
	budget	104	17.3
	unknown	95	15.8
	competitor	92	15.3
	pricing	91	15.2



Is churn getting better or worse over time?

FINDING: Churn is accelerating

Early 2024: Low churn events

Mid 2024: Moderate churn events

Q4 2024: Highest churn events

Early users stayed longer, they were excited by the new product. As the product failed to deliver new features and fast support, more users started leaving. The longer RavenStack waits to fix the core problems, the worse this trend will become after public launch.

```sql
select
    date_format(churn_date, '%Y-%m') as month,
    count(*) as churn_events
from ravenstack_churn_events
where churn_date is not null
group by DATE_FORMAT(churn_date, '%Y-%m')
order by month;
```

OUTPUT

	month	churn_events
	2023-12	16
	2024-01	20
	2024-02	10
	2024-03	24
	2024-04	25
	2024-05	27
	2024-06	40
	2024-07	35
	2024-08	42
	2024-09	53
	2024-10	66
	2024-11	68
	2024-12	117

The Support Crisis Hidden in Plain Sight

Average satisfaction: 2.4 out of 5
Across ALL 500 accounts — not just churned ones

- Initial hypothesis:

Churned accounts = worse support experience

- Actual finding:

Both groups show equally poor satisfaction
2.41 vs 2.32 — almost no difference

- The real story:

Support quality is universally bad
This is not a churn segment problem
This is a company-wide crisis

Going public with 2.4 satisfaction will accelerate churn not slow it

```sql
select
    case
        when a.churn_flag_clean = 1 then 'Churned'
        else 'Retained'
    end as customer_status,
    count(t.ticket_id) as total_tickets,
    round(avg(t.satisfaction_score), 2) as avg_satisfaction
from ravenstack_accounts a
join ravenstack_support_tickets t
    on a.account_id = t.account_id
group by a.churn_flag_clean
order by a.churn_flag_clean desc;
```

OUTPUT:

customer_status	total_tickets	avg_satisfaction
Churned	432	2.41
Retained	1568	2.32

Who Is Leaving Next?

20 active accounts identified Showing same warning signs as churned users Still here, window to save them is open Action needed: This week before launch

```sql
select a.account_id, a.industry, a.plan_tier,
count(distinct t.ticket_id) as total_tickets,
round(avg(t.satisfaction_score), 1) as avg_satisfaction,
round(sum(s.mrr_amount), 0) as total_mrr
from ravenstack_accounts a
join ravenstack_support_tickets t
on a.account_id = t.account_id
join ravenstack_subscriptions s
on a.account_id = s.account_id
where a.churn_flag_clean = 0
and s.is_trial_clean = 0
and s.mrr_amount > 0
group by a.account_id, a.industry, a.plan_tier
having avg_satisfaction < 3.5
and total_tickets >= 5
order by total_mrr desc
limit 20;
```

account_id	industry	plan_tier	total_tickets	avg_satisfaction	total_mrr
A-e08cd3	FinTech	Pro	8	2	568160
A-7920cc	Cybersecurity	Pro	7	1.9	364623
A-fa3095	FinTech	Basic	5	2	346150
A-1f7acb	FinTech	Basic	7	3.1	328384
A-ed510f	Cybersecurity	Basic	9	1.8	325458
A-86902e	HealthTech	Enterprise	9	1.6	324369
A-40906c	EdTech	Basic	5	3	322000
A-1e1b80	FinTech	Enterprise	6	2.7	291882
A-dbc825	EdTech	Enterprise	6	1.3	254424
A-e43bf7	Cybersecurity	Pro	5	2.4	233330
A-2e3bad	FinTech	Pro	5	1.4	229360
A-8b0451	FinTech	Enterprise	7	2	221963
A-158f4e	FinTech	Enterprise	5	2.6	219295
A-82861f	DevTools	Basic	5	3.2	218165
A-883b7d	Cybersecurity	Basic	7	2.6	213668
A-671f31	DevTools	Basic	7	3	212695
A-9f9299	FinTech	Enterprise	5	1	212015
A-7e0a10	FinTech	Enterprise	5	1.6	205580
A-c70870	Cybersecurity	Basic	5	2.4	198070
A-526d93	HealthTech	Basic	5	1.4	196005

THANK YOU